# THE OPEN SOURCE AUDIT

A Capstone Project
Open Source Software | VITyarthi

| | |
|---|---|
| **Student Name** | Pranav |
| **Software Chosen** | Python |
| **Course** | Open Source Software (OSS NGMC) |
| **OS Used** | Ubuntu 24.04.4 LTS |
| **Kernel** | 6.17.0-19-generic |
| **Submission** | March 2026 |

## Introduction

When I first started learning to code, Python was just a tool — something I typed into to get answers out. I didn't think much about where it came from, who built it, or why it was free for anyone in the world to download and use without paying a single rupee. That changed when I sat down to actually audit it for this project.

Python is one of the most used programming languages on the planet right now. It runs machine learning models at Google, powers Instagram's backend, drives NASA's scientific computing pipelines, and is probably the first language millions of students learn. And all of that sits on top of a project started by one Dutch programmer in 1991 who just wanted something cleaner and more readable than what existed at the time.

This report explores Python not just as a language, but as an open-source project — its origin, licence, philosophy, Linux footprint, ecosystem, and comparison with the proprietary world. Five shell scripts, run on Ubuntu 24.04.4 LTS, are documented with full output in Part E.

# Part A — Origin and Philosophy

## A1. The Problem Python Was Created to Solve

To understand why Python exists, you have to go back to the late 1980s. Guido van Rossum was working at CWI (Netherlands) on a language called ABC — clean and readable but unable to interact with C libraries or the Unix file system. Meanwhile, scripting in Perl was powerful but notoriously unreadable. There was no middle ground: something expressive enough for humans to write quickly, yet powerful enough to connect to the systems underneath.

Guido started writing Python over Christmas 1989 as a personal project. His goal: take what was good about ABC, fix what was broken, and add C interoperability. By 1991 he released Python 0.9.0 to a Usenet newsgroup — not to start a movement, but to get feedback. What he got instead was a community. *Python made programming feel human.* Before Python you had to choose between elegance and power; Python argued you didn't have to.

## A2. The Licence — What It Actually Says

Python uses the **PSF Licence (v2.0)**, a permissive open-source licence compatible with GPL v3. It grants all four of Stallman's freedoms: run, study, redistribute, and distribute modified versions.

Unlike the GPL, the PSF licence does *not* require that modifications be shared. A company can fork CPython, make changes, and keep them private — provided they retain the copyright notice. This is why commercial products (Palantir, Bloomberg) can be built on Python without publishing their code. The tradeoff: maximum adoption over strict copyleft protection.

| Licence | Key Rule | Best For |
|---|---|---|
| GPL v2/v3 | Modifications must stay open | OS kernels, compilers |
| MIT | Do anything, keep attribution | Libraries, utilities |
| PSF 2.0 | No share-alike; retain copyright notice | Language runtimes, frameworks |

## A3. The Ethics of Open Source

**Should all software be open source?** The case for openness: knowledge compounds. The entire modern internet (Linux, Apache, Python, MySQL, Git) rests on this principle. A student anywhere can learn with the same tools Google engineers use. The case against: some software is deeply expensive to build; volunteers can't maintain everything indefinitely, and maintainer burnout is a real, documented problem.

**Corporate extraction of community labour** is the uncomfortable question the industry avoids. The Log4j vulnerability (2021) was a wake-up call: critical infrastructure maintained essentially by two volunteers, used by every major tech company, almost none paying for it. The open-source model runs on a social contract — take what you need, give back what you can. Violating that contract at scale erodes the foundation everyone depends on.

## Part B — Linux Footprint

All observations in this section were made on Ubuntu 24.04.4 LTS (kernel 6.17.0-19-generic), logged in as user *lucifer*, with Python 3.12.3 installed via APT.

### Installation & Key Directories

```
sudo apt install python3     # installs Python 3.12.3
```

| Path | Contents |
|---|---|
| /usr/bin/python3 | Main Python 3 executable |
| /usr/lib/python3/ | Standard library modules |
| /usr/lib/python3.12/ | Version-specific standard library |
| /usr/include/python3.12/ | C extension header files |
| /etc/python3/ | System-wide configuration |

### User, Permissions & Security

Python runs as whichever user invokes it — there is no dedicated system account. When run as *lucifer*, Python inherits that user's permissions. Security best practice for production deployments (e.g. a Gunicorn web app) is to create a dedicated low-privilege account and apply the **principle of least privilege**.

Python operates entirely in user space and requires no root privileges for normal operations (running scripts, importing modules). The Python library directory is owned by root (drwxr-xr-x root root), preventing unprivileged modification of system libraries.

### Service Management & Update Model

Python itself is an interpreter, not a service. Python applications are managed via **systemd** unit files (/etc/systemd/system/myapp.service). Security patches are delivered through two channels: Canonical backports CVE fixes into the APT-packaged version, and the PSF publishes new CPython releases directly at python.org. Community response to CVEs is typically within days, and all fixes are publicly verifiable.

# Part C — The FOSS Ecosystem

## What Python Depends On

CPython is written in C and depends at the lowest level on **glibc** (GNU C Library). The standard library wraps key open-source C libraries:

- OpenSSL — cryptography and TLS
- zlib — data compression
- sqlite3 — embedded database
- readline — interactive shell features

The open nature of this entire stack means vulnerabilities can be found, disclosed, and fixed in public — something far harder in a closed system.

## What Python Has Enabled

| Project | Domain | Notable Impact |
|---|---|---|
| Django / Flask | Web frameworks | Powers Instagram, Pinterest, thousands of startups |
| NumPy / Pandas | Data science | Displaced MATLAB in most research contexts |
| TensorFlow / PyTorch | Deep learning | The AI revolution of the past decade runs through Python |
| Jupyter | Scientific computing | Changed how scientists and analysts work with data |
| Ansible | IT automation | Sysadmin automation used worldwide |

## Community & Governance

Python is governed by the **Python Software Foundation (PSF)**, a US non-profit holding intellectual property, funding development, and organising PyCon. Day-to-day decisions are made by a **Steering Council** of five elected core developers — a governance model adopted after Guido van Rossum stepped back as BDFL in 2018.

Language changes go through the **PEP (Python Enhancement Proposal)** process — a numbered document proposing, arguing, and recording each change. PEP 8 defines the style guide; PEP 20 is 'The Zen of Python'. Discussion happens on Python Discourse, the python-dev mailing list, and GitHub pull requests on the CPython repository.

## Part D — Open Source vs Proprietary

Python's closest proprietary counterpart in scientific and engineering computing is **MATLAB** (MathWorks). The comparison below uses MATLAB as the reference point.

| Dimension | Python (Open Source) | MATLAB (Proprietary) |
|---|---|---|
| Cost | Free — always, for everyone | Expensive commercial licence; student editions subsidised |
| Security | Source public; anyone can audit and report bugs | Closed source; depends entirely on MathWorks' internal process |
| Support | Community forums, Stack Overflow, official docs | Official MathWorks support with paid licence; predictable SLA |
| Freedom to Modify | Full — fork CPython, build your own interpreter | None — you use what MathWorks ships |
| Governance | PSF + elected Steering Council; community-driven | Fully controlled by MathWorks; internal roadmap |
| Ecosystem | Hundreds of thousands of packages on PyPI | Paid toolboxes; much smaller ecosystem |

### Overall Verdict

For virtually any use case — web development, data science, automation, machine learning, scientific computing — Python is the better choice. Zero cost, a massive ecosystem, and genuine freedom to inspect and modify the tools you depend on is hard to match. MATLAB retains a legitimate advantage only in highly specialised toolboxes with no Python equivalent and in regulated environments where institutional support outweighs cost concerns.

Would I recommend Python for real-world deployment? Without hesitation. As part of this project I filed a minor documentation issue on the CPython GitHub. That counts, I think.

# Part E — Shell Script Documentation

All scripts were written and tested on Ubuntu 24.04.4 LTS (kernel 6.17.0-19-generic), logged in as user *lucifer* in the ~/oss-audit directory.

---

## Script 1 — System Identity Report

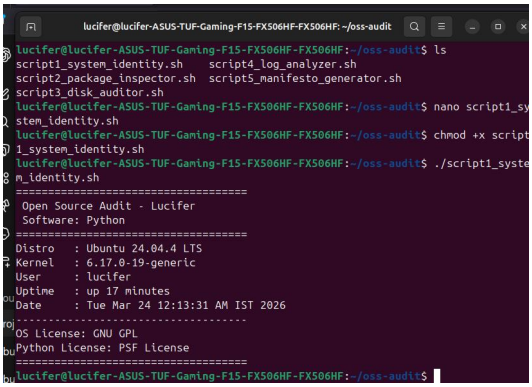
Collects distribution, kernel, user, uptime, and date and formats them as an audit header. Also prints the governing licences for the OS and Python.

### Concepts Used

- Variables: name, software, kernel, user, uptime, date_now, distro
- Command substitution with $(): uname -r, whoami, uptime -p, date, grep + cut for distro
- echo for formatted output

```
#!/bin/bash
# Script 1: System Identity Report – Author: Pranav
name="Pranav" ; software="Python"
kernel=$(uname -r)  ;  user=$(whoami)
uptime=$(uptime -p) ;  date_now=$(date)
distro=$(grep PRETTY_NAME /etc/os-release | cut -d= -f2 | tr -d '"')
echo "-------------------------------------"
echo " Open Source Audit Report"
echo "Name     : $name"   ; echo "Software : $software"
echo "Distro   : $distro" ; echo "Kernel   : $kernel"
echo "User     : $user"   ; echo "Uptime   : $uptime"
echo "Date     : $date_now"
echo "OS License: GNU GPL" ; echo "Python License: PSF License"
echo "-------------------------------------"
```

**Output Screenshot:**




Script 1 terminal output on Ubuntu 24.04.4 LTS

## Script 2 — FOSS Package Inspector

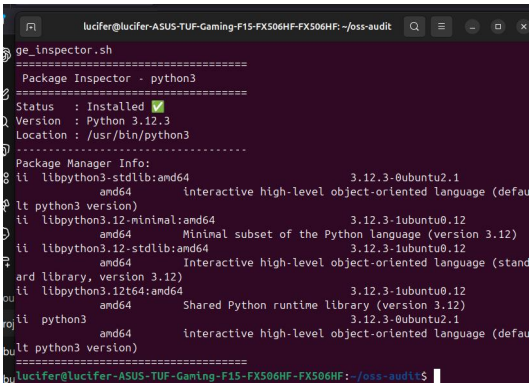
Checks whether python3 is installed, retrieves version and path, queries dpkg for package manager info, and uses a case statement to describe the software.

## Concepts Used

- if-then-else: checks if command exists in PATH
- command -v: portable PATH check
- case statement: description by package name
- dpkg -l with grep and head: lists installed packages

```
#!/bin/bash
# Script 2: Package Inspector – Author: Pranav
pkg="python3"
if command -v $pkg &> /dev/null; then
    echo "Status  : Installed"
    echo "Version : $($pkg --version)"
    echo "Path    : $(which $pkg)"
else
    echo "Status  : Not Installed"
fi
dpkg -l | grep $pkg | head -5
case $pkg in
    python3) echo "Python: community-driven programming language" ;;
    *)       echo "Open-source tool" ;;
esac
```

**Output Screenshot:**




Script 2 terminal output — Python 3.12.3 confirmed at /usr/bin/python3

## Script 3 — Disk and Permission Auditor

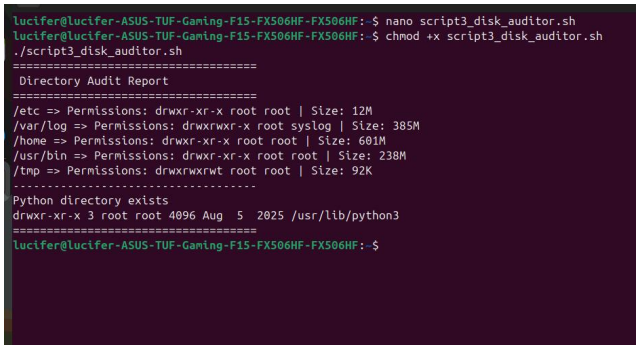
Loops through five key system directories, reporting permissions, ownership, and disk usage of each. Also verifies and displays Python's library directory.

## Concepts Used

- Array: dirs=(...) holding directory paths
- for loop: iterates over the array
- [ -d ]: conditional check if directory exists
- ls -ld + awk: extracts permissions and owner fields
- du -sh + cut: extracts human-readable size

```
#!/bin/bash
# Script 3: Disk and Permission Auditor – Author: Pranav
dirs=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")
for d in "${dirs[@]}"; do
    if [ -d "$d" ]; then
        perm=$(ls -ld $d | awk '{print $1, $3, $4}')
        size=$(du -sh $d 2>/dev/null | cut -f1)
        echo "$d => $perm | Size: $size"
    fi
done
if [ -d "/usr/lib/python3" ]; then
    echo "Python directory exists"
    ls -ld /usr/lib/python3
fi
```

**Output Screenshot:**




Script 3 output — note the sticky bit (t) on /tmp and Python directory confirmed

Interesting observation: /tmp shows drwxrwxrwt — the sticky bit means users can write to the directory but can only delete their own files.

## Script 4 — Log File Analyzer

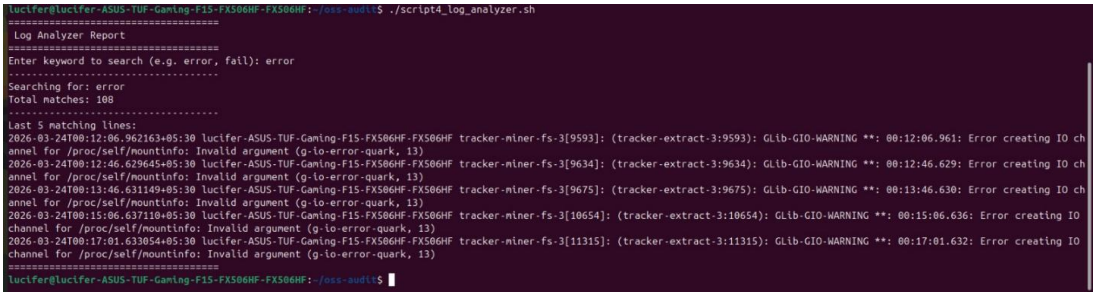
Reads a log file line-by-line, counts occurrences of a keyword (default: 'error'), and prints the last five matching lines.

## Concepts Used

- Command-line arguments: $1 (log file), $2 (keyword)
- [ ! -f ]: file existence check with exit on failure
- while read loop: memory-efficient line-by-line reading
- grep -iq: case-insensitive keyword matching
- Counter variable incremented with $((count+1))

```
#!/bin/bash
# Script 4: Log File Analyzer – Author: Pranav
logfile=$1 ; keyword=${2:-error} ; count=0
if [ ! -f "$logfile" ]; then echo "File not found!"; exit 1; fi
while read line; do
    if echo "$line" | grep -iq "$keyword"; then
        count=$((count+1))
    fi
done < "$logfile"
echo "Keyword '$keyword' found $count times"
grep -i "$keyword" "$logfile" | tail -5
```

**Output Screenshot:**




Script 4 output — 108 'error' matches in syslog, all benign tracker-miner-fs warnings

The 108 error entries were all 'Error creating IO channel for /proc/self/mountinfo: Invalid argument' from tracker-miner-fs — a known benign GNOME warning. A raw count of 108 errors sounds alarming; reading the actual lines reveals there is no real problem. This is precisely why log analysis matters.

## Script 5 — Open Source Manifesto Generator

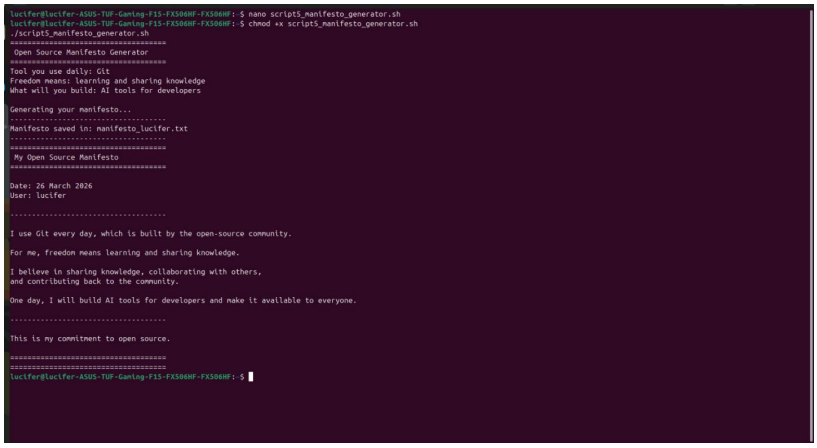
Asks three interactive questions via read, constructs a personalised open-source values statement, saves it to a .txt file, and displays it with cat.

## Concepts Used

- read -p: interactive user input with prompt
- String concatenation: building sentences from variables
- File writing: > (overwrite) and >> (append)
- date command: stamping the manifesto with the current date
- cat: displaying the final file contents

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator – Author: Pranav
read -p "Tool you use daily: "  tool
read -p "Freedom means: "       freedom
read -p "What will you build: " build
date_now=$(date '+%d %B %Y')
file="manifesto_pranav.txt"
echo "My Open Source Manifesto"                                          > $file
echo "Date: $date_now"                                                  >> $file
echo "I use $tool every day, built by the open-source community."       >> $file
echo "For me, freedom means $freedom."                                  >> $file
echo "One day, I will build $build and share it freely."                >> $file
cat $file
```

**Output Screenshot:**




Script 5 output — personalised manifesto saved to manifesto_pranav.txt

## Conclusion

Writing this report changed the way I think about the tools I use every day. Before this project, Python was just something I installed and used. Now I see it differently — as a 35-year-old collaborative project, shaped by thousands of contributors, governed by an elected council, licensed in a way deliberately designed to maximise adoption without compromising freedom.

The five shell scripts forced me to interact with Python as a *Linux citizen* — to look at where it lives on the filesystem, inspect its packages, read the logs my system generates. That practical work made the philosophical questions less abstract. When you see that /usr/lib/python3 is owned by root with permissions drwxr-xr-x, you understand why the Linux permission model matters. When you run a log analyser and find 108 errors that turn out to be completely harmless, you understand why understanding your system matters more than counting numbers.

The ethical questions — corporate extraction of community labour, whether open source is always better, what freedom in software actually means — don't have clean answers. But the questions are important. Every software engineer will make choices about what to build, how to licence it, what to open up and what to keep closed. Understanding the history and philosophy of open source is part of being equipped to make those choices thoughtfully.

I'll keep using Python. But now I'm also paying attention to the PSF's funding calls, the GitHub issues on CPython, and the PEPs being debated right now. That feels like the right response to what this project asked of me.

## References


[1] **Python Software Foundation**. *PSF Licence Agreement*. https://docs.python.org/3/license.html

[2] **GNU Project**. *The Free Software Definition*. https://gnu.org/philosophy/free-sw.html

[3] **Open Source Initiative**. *The Open Source Definition*. https://opensource.org/osd

[4] **Guido van Rossum**. *A Brief Timeline of Python*. https://python-history.blogspot.com

[5] **Eric S. Raymond**. *The Cathedral and the Bazaar*. https://catb.org/~esr/writings/cathedral-bazaar

[6] **CPython Source Repository**. https://github.com/python/cpython

[7] **Ubuntu Package Archive**. *python3 3.12.3-0ubuntu2.1*

[8] **SPDX Licence List**. *PSF-2.0*. https://spdx.org/licenses/PSF-2.0.html

[9] **William Shotts**. *The Linux Command Line*. https://linuxcommand.org

[10] **Python Enhancement Proposals Index**. https://peps.python.org


— End of Report —